ConfiguraçãoInicialUbuntuClient

Nov 8, 2026 · @Orlando

Objetivo

No fim deste guião, o Ubuntu Client tem IP estático, usa o Windows Server como DNS e responde a `ping isec.local`, sem conflitos de rede.

Este é o ponto de partida da integração: sem o DNS apontado ao Windows, o Kerberos e o SSSD não conseguem descobrir o domínio.

Atenção: todos os endereços IP deste guião dependem da *pool de endereços* do teu grupo. Os exemplos usam a pool **105** (rede 192.168.105.0/24). **Substitui sempre os endereços 192.168.X.Y pelos da tua pool.**

Antes de começar: valores a adaptar

Os comandos usam o formato 192.168.X.Y e o nome de ligação `labX`. X é o terceiro número da tua pool (na pool 105, X = 105).

O que é	Formato	Exemplo (pool 105)
IP do Ubuntu Client	192.168.X.Y	192.168.105.3
Máscara de rede	/24	/24
Gateway (saída da rede)	192.168.X.Y	192.168.105.1
IP do Windows Server (também o DNS)	192.168.X.Y	192.168.105.2
Domínio	isec.local	isec.local
Nome da ligação	labX	lab105
Placa de rede	ens18	ens18

Confirma estes valores na tua pool antes de começar. **Não copies os exemplos sem os trocar pelos da tua pool.** O nome da placa pode variar: confirma-o com `ip -br a`.

Requisitos prévios

1. Ubuntu e Windows Server clonados do template, com as dependências já instaladas (Kerberos, SSSD, Samba).

2. As duas VMs **na mesma bridge** (a `vmbrX` da tua pool). Depois de clonar, confirma no Proxmox, em Hardware → Network Device de cada VM: mesma Bridge, com Disconnect e Firewall desmarcados.
3. Windows Server com IP estático, domínio `isec.local` criado e uma regra de firewall que permita ping vindo do IP do Ubuntu.
4. Trabalha na consola da VM, porque a rede vai ser reconfigurada e uma sessão SSH cairia.

Passo 1: Ver a configuração de rede existente

Objetivo: perceber que ficheiros e ligações já configuram a placa, para evitar conflitos.

```
ls /etc/netplan
nmcli -f NAME,DEVICE,TYPE connection show
```

- `ls /etc/netplan` lista os ficheiros do Netplan, o sistema que define a rede no Ubuntu.
- `nmcli … connection show` lista as ligações geridas pelo NetworkManager.

Porquê: o erro *Conflicting default route* aparece quando mais de um ficheiro ou perfil configura a mesma placa (`ens18`) com uma rota por defeito.

Captura: o output, com os ficheiros e os perfis existentes.

Passo 2: Criar uma pasta de backup

Objetivo: guardar a configuração antiga em vez de a apagar, para poder repor se algo correr mal.

```
sudo mkdir -p /root/netplan-bak
```

`mkdir` cria a pasta. `-p` não dá erro se ela já existir. A pasta fica em `/root` , só acessível ao administrador.

Atenção: se saltares este passo, o Passo 3 falha com *Ficheiro ou pasta inexistente*.

Passo 3: Retirar os ficheiros em conflito

Objetivo: deixar um único método de configuração de rede, o NetworkManager por linha de comandos.

```
sudo mv /etc/netplan/00-installer-config.yaml /etc/netplan/01-netcfg.yaml
/etc/netplan/90-NM-*.yaml /root/netplan-bak/
ls /etc/netplan
```

- `mv` move os três ficheiros que configuram a placa para a pasta de backup.
- Fica apenas o `01-network-manager-all.yaml`, que só diz que o NetworkManager gere a rede e não configura nenhuma placa.

Resultado esperado: o `ls` mostra só `01-network-manager-all.yaml`.

Nota: os nomes dos ficheiros podem variar. A regra é mover todos, exceto o `01-network-manager-all.yaml`.

Captura: o `ls` final.

Passo 4: Aplicar a limpeza

Objetivo: fazer o Ubuntu voltar a ler os ficheiros de rede, já sem a configuração antiga.

```
sudo netplan apply
```

Como já não há ficheiro a configurar a `ens18`, o perfil antigo desaparece.

Nota: podem aparecer avisos amarelos sobre o `systemd-networkd`. São inofensivos, porque não o usamos.

Passo 5: Garantir que o NetworkManager está a correr

Objetivo: confirmar que o serviço que gere a rede por comandos está ativo.

```
sudo systemctl start NetworkManager
systemctl is-active NetworkManager
```

- `systemctl start` arranca o serviço (não faz mal se já estiver a correr).
- `is-active` confirma o estado.

Resultado esperado: `active`.

Porquê: sem o serviço ativo, os comandos `nmcli` falham com *O NetworkManager não está em execução*, e a ligação do Passo 6 nunca é criada.

Captura: o `active`.

Passo 6: Criar a ligação com IP estático e DNS do Windows

Objetivo: dar ao Ubuntu um IP fixo e apontar o DNS para o Windows Server.

Atenção: troca os endereços 192.168.X.Y e o nome `labX` pelos da tua pool. Cada linha 192.168.X.Y é um endereço diferente, identificado na tabela abaixo.

```
sudo nmcli connection add type ethernet ifname ens18 con-name labX \
  ipv4.method manual \
  ipv4.addresses 192.168.X.Y/24 \
  ipv4.gateway 192.168.X.Y \
  ipv4.dns 192.168.X.Y \
  ipv4.dns-search isec.local \
  ipv4.ignore-auto-dns yes
```

Parâmetro	O que faz	Valor	Exemplo (pool 105)
`ifname ens18`	Placa de rede a configurar	nome da tua placa	ens18
`con-name labX`	Nome do perfil	labX	lab105
`ipv4.method manual`	IP fixo, sem DHCP	manual	manual
`ipv4.addresses`	IP do Ubuntu e máscara	192.168.X.Y/24	192.168.105.3/24
`ipv4.gateway`	Saída da rede	192.168.X.Y	192.168.105.1
`ipv4.dns`	**Redirecionamento do DNS para o Windows Server**	192.168.X.Y	192.168.105.2
`ipv4.dns-search`	Domínio de pesquisa (escreve `SR2` em vez de `SR2.isec.local`)	isec.local	isec.local
`ipv4.ignore-auto-dns yes`	Impede que outro DNS entre automaticamente	yes	yes

Porquê só o DNS do Windows: não ponhas `8.8.8.8` como alternativo. O Ubuntu pode perguntar-lhe e não encontrar os registos internos do AD. A internet continua a funcionar através do *forwarder* configurado no DNS do Windows.

Captura: o comando executado, sem erros.

Passo 7: Ativar a ligação e confirmar

Objetivo: ligar o perfil criado e ver que a placa o está a usar. Usa o mesmo nome do Passo 6 (`labX` , por exemplo `lab105`).

```
sudo nmcli connection up labX
nmcli -f NAME,DEVICE,TYPE connection show
```

- `connection up` liga o perfil.
- `connection show` lista as ligações.

Resultado esperado: o perfil `labX` aparece associado à placa `ens18` .

Nota: se aparecer outro perfil antigo sem placa (por exemplo *Ligação com fios 1*), apaga-o com `sudo nmcli connection delete 'NOME'` , para não confundir.

Captura: o output final.

Passo 8: Confirmar o IP e a rota

Objetivo: ver que a placa tem o IP certo e uma única rota por defeito.

```
ip -br a
ip route
```

- `ip -br a` mostra o IP de cada placa, em formato resumido.
- `ip route` mostra as rotas da máquina.

Resultado esperado (exemplo da pool 105):

- A `ens18` aparece `UP` com `192.168.105.3/24` , e já não com um endereço `169.254…` . Um endereço `169.254` significa que o IP estático não foi aplicado.
- Existe **uma única** linha `default via 192.168.105.1` . Na tua pool, será `default via 192.168.X.Y` , com o gateway que usaste no Passo 6.

Porquê: uma só rota por defeito prova que o conflito do início desapareceu.

Captura: o output dos dois comandos.

Passo 9: Confirmar que o DNS é o do Windows

Objetivo: ver qual o servidor DNS que o Ubuntu usa nesta placa.

```
resolvectl status ens18
```

Resultado esperado (exemplo da pool 105):

- `Current DNS Server: 192.168.105.2`
- `DNS Servers: 192.168.105.2`, **sem** `8.8.8.8` nem `1.1.1.1`
- `DNS Domain: isec.local`

Na tua pool, o DNS tem de ser o IP do Windows Server (192.168.X.Y) que usaste no Passo 6.

Porquê: se ainda aparecerem DNS públicos, o Ubuntu pode perguntar a um deles e falhar a descoberta do domínio. Volta ao Passo 6 e confirma o `ipv4.ignore-auto-dns yes`.

Captura: o output do `resolvectl`.

Passo 10: Testar a ligação ao servidor

Objetivo: confirmar que o Ubuntu chega ao Windows Server pelo IP. Troca 192.168.X.Y pelo IP do **Windows Server** da tua pool (exemplo: 192.168.105.2).

```
ping -c 3 192.168.X.Y
```

`ping -c 3` envia 3 pacotes e pára.

Resultado esperado: `0% packet loss`.

Porquê: prova que há rede, mas ainda não que o DNS funciona. O IP não precisa de DNS.

Se falhar:

- `Destination Host Unreachable` e `ip neigh show` com `INCOMPLETE` ou `FAILED`: o Ubuntu não vê o Windows. Quase sempre as duas VMs **não estão na mesma bridge** no Proxmox. Confirma em Hardware → Network Device de cada VM.
- Se o Windows aparece mas o ping é bloqueado: falta a regra de firewall do Windows que permite ping vindo do IP do Ubuntu.

Captura: o ping com 0% de perda.

Passo 11: Testar o DNS do domínio

Objetivo: provar que o DNS do Windows responde sobre o domínio e anuncia o serviço de autenticação. Troca 192.168.X.Y pelo IP do **Windows Server** da tua pool.

```
dig @192.168.X.Y isec.local +short
dig @192.168.X.Y _ldap._tcp.dc._msdcs.isec.local SRV +short
```

- O primeiro pergunta ao Windows qual é o IP de `isec.local`.
- O segundo pede o registo **SRV**, que diz onde está o serviço de autenticação (LDAP) do domínio.

Resultado esperado (exemplo da pool 105):

- O primeiro devolve `192.168.105.2`.
- O segundo devolve algo como `0 100 389 nome-do-servidor.isec.local.`

Porquê: o registo SRV é a prova mais importante. É com ele que o Kerberos e o SSSD descobrem o controlador de domínio.

Captura: o output dos dois comandos.

Passo 12: Corrigir o ping isec.local

Objetivo: fazer o Ubuntu perguntar ao DNS do Windows quando o nome acaba em `.local`.

```
sudo cp /etc/nsswitch.conf /etc/nsswitch.conf.bak
sudo sed -i 's/mdns4_minimal \[NOTFOUND=return\] //' /etc/nsswitch.conf
grep ^hosts /etc/nsswitch.conf
```

- A 1.ª linha guarda uma cópia do ficheiro original.
- A 2.ª remove `mdns4_minimal [NOTFOUND=return]` da linha `hosts` do ficheiro `/etc/nsswitch.conf`, que define por que ordem o Ubuntu resolve nomes.
- A 3.ª mostra o resultado.

Porquê: por defeito, o Ubuntu trata nomes `.local` como mDNS (descoberta na rede local) e responde logo "nome desconhecido", sem nunca perguntar ao DNS do Windows. Por isso o `dig` do Passo 11 funciona e o `ping isec.local` falha.

Resultado esperado: a linha passa de `hosts: files mdns4_minimal [NOTFOUND=return] dns` para `hosts: files dns`.

Captura: o `grep`, de preferência antes e depois.

Passo 13: Teste final, ping isec.local

Objetivo: confirmar que o domínio resolve pelo nome.

```
getent hosts isec.local
ping -c 3 isec.local
```

- `getent hosts` mostra como o sistema resolve o nome, exatamente como o `ping` o faz.
- `ping isec.local` testa o domínio pelo nome.

Resultado esperado (exemplo da pool 105): o `getent` devolve `192.168.105.2 isec.local` e o ping responde com `0% packet loss`. Na tua pool, o IP será o do Windows Server (192.168.X.Y).

Captura: o `ping isec.local` a responder. É a prova principal desta fase.

Erros comuns e como resolver

Sintoma	Causa provável	Solução
`Conflicting default route` no `netplan apply`	Mais de um ficheiro ou método a configurar a mesma placa	Passos 1 a 4: ficar com um único método
`mv: destino '/root/netplan-bak/': Ficheiro ou pasta inexistente`	Faltou criar a pasta de backup	Passo 2 e repetir o Passo 3
`O NetworkManager não está em execução`	Serviço parado	Passo 5 e repetir o Passo 6
IP `169.254.x.x` na `ens18`	O IP estático não foi aplicado	Passos 6 e 7
`Destination Host Unreachable`, `ip neigh` com `INCOMPLETE` ou `FAILED`	As VMs não estão na mesma bridge, ou a placa tem Disconnect ou Firewall marcados	Proxmox, Hardware → Network Device das duas VMs
O ping ao IP do Windows falha, mas o MAC aparece no `ip neigh`	Firewall do Windows a bloquear o ping	Regra de ICMPv4 para o IP do Ubuntu
`dig` funciona e `ping isec.local` falha	`mdns4_minimal` no `nsswitch.conf`	Passo 12
Sem internet com o DNS só no Windows	O DNS do Windows não tem *forwarder*	No Windows: `Add-DnsServerForwarder -IPAddress 1.1.1.1`

Resumo e passo seguinte

No fim, o Ubuntu Client tem um único perfil de rede (`labX`) com IP estático, usa o Windows Server como único DNS e resolve `isec.local` pelo nome. Esta configuração é a base de tudo o que se segue, porque o Kerberos e o SSSD descobrem o domínio através deste DNS.

Próximos passos do plano: sincronizar o relógio com o servidor (o Kerberos rejeita diferenças superiores a 5 minutos), configurar o `krb5.conf` e validar com `kinit` e `klist`. Só depois vêm o Samba e o SSSD.